



BIRC 商智中心

Business Intelligence Center — 推動校園數位人才培育

2026 後端群體驗營

```
$ java OrderSystem --mode interactive
```

Backend 體驗營 Day2

// 訂單系統 — 從陣列到「真實世界的後端」

平行陣列
組織多筆資料

庫存判斷
避免超賣的關鍵

訂單邏輯
購物車的靈魂



昨天回顧

Day 1 學了什麼

- ✓ 變數 — 存東西的盒子
- ✓ 條件判斷 — if/else 分支
- ✓ 迴圈 — while 重複執行
- ✓ Scanner — 讀取輸入



今天要做的事

-  用平行陣列存商品資料
-  做一個超簡化的購物車
-  加入庫存判斷（避免超賣）



重點：同樣的工具（變數、if/else、while、陣列），可以做出完全不同的東西！

今日目標



做出一個可互動的訂單系統

商品清單 + 選購 + 結帳



理解「庫存扣減」的邏輯

買到剩 0 件就不能再買



把資料放進真正的資料庫

建立資料表 + 新增資料 + 查詢資料



購物車的靈魂

你點外送 App 按下「加入購物車」後，後端發生了什麼？

1. 讀取商品

從資料庫拿商品清單



2. 檢查庫存

還有貨嗎？



3. 扣庫存

把它從架上拿掉



4. 計算金額

加總所有商品



這些都是後端 — 你看不到，但它一直在運作



步驟一：陣列就是資料的容器

第一部 — 先用陣列保存商品資料

```
1 // 陣列：把多筆商品資料暫時放在程式記憶體裡
2 String[] products = {"cola", "sandwich", "chips", "cookie", "juice"};
3 int[] prices = {30, 65, 45, 35, 40};
4
5 // 印出商品選單
6 for (int i = 0; i < products.length; i++) {
7     System.out.println((i + 1) + ". " + products[i] + " → " + prices[i] + " 元");
8 }
9
10 // 讓使用者選購
11 System.out.print("請輸入想買的商品編號 (1-" + products.length + ") : ");
12 int choice = scanner.nextInt();
```

陣列在做什麼？

它就是目前存放商品資料的地方：名稱放在 `products`，價格放在 `prices`。

它的限制是什麼？

程式結束後資料就消失，而且資料筆數與查詢能力都有限。

1 2
3 4

陣列索引 Index 是什麼？

Step 1.5 — 陣列的「門牌號碼」

```
1 // 陣列的 index 從 0 開始！
2 String[] products = {"cola", "sandwich", "chips", "cookie", "juice"};
3
4 //           ↑       ↑       ↑       ↑       ↑
5 //   index:      0       1       2       3       4
6 //   products[0] = "cola"
7 //   products[1] = "sandwich"
8 //   products[4] = "juice"    ← 最後一個是 length - 1
```

⚠ 常見錯誤

products[5] →

ArrayIndexOutOfBoundsException

index 最大只能是 length - 1

💡 為什麼從 0 開始？

index 代表「距離起點幾個位置」

第 1 個元素 = 距離起點 0 個位置



你剛剛用了什麼？



平行陣列

products + prices

同一 index = 同一商品



For 迴圈

```
for (int i = 0; ...)
```

印出所有商品



條件判斷

```
if (choice >= 1 && ...)
```

確認輸入範圍



重點：你昨天學的工具，今天馬上用上了！



步驟二：多商品訂單

第一部 — 用陣列完成訂單邏輯

```
1  int totalAmount = 0; // 總金額
2  int orderCount = 0;  // 買了幾個
3  while (true) {
4      System.out.println("商品列表：");
5      for (int i = 0; i < products.length; i++) {
6          System.out.println((i + 1) + ". " + products[i] + " → " + prices[i] + " 元");
7      }
8      System.out.println("0. 結帳");
9      System.out.print("請選擇商品編號：");
10     int choice = scanner.nextInt();
11
12     if (choice == 0) {
13         break; // 結帳跳出迴圈
14     }
15
16     if (choice >= 1 && choice <= products.length) {
17         System.out.print("買幾個？");
18         int quantity = scanner.nextInt();
19         int subtotal = prices[choice - 1] * quantity;
20         totalAmount += subtotal;
21         orderCount += quantity;
22     }
23 }
```



迴圈的妙用



while (true) 迴圈

- • 不斷顯示商品選單
- • 讓使用者可以一直買
- • 輸入 0 才跳出

→ 這就是「購物車」的迴圈！



累加金額

- • `totalAmount += subtotal`
- • 每買一個就加一次
- • 最後就是總金額

→ 這就是「結帳」的計算！



重點：迴圈讓使用者可以一直買，`break` 讓使用者可以結帳



步驟三：庫存判斷（重點）

第一部 — 陣列裡也可以保存庫存

```
1  int[] stock = {10, 5, 8, 12, 7}; // 庫存數量
2
3  // 檢查庫存
4  if (stock[index] == 0) {
5      System.out.println("❌ " + products[index] + " 已售完，無法購買！");
6      continue; // 跳過這次，回到迴圈開頭
7  }
8  // 庫存不足
9  if (quantity > stock[index]) {
10     System.out.println("❌ 庫存不足！只剩 " + stock[index] + " 個");
11     continue;
12 }
13 // 扣庫存
14 stock[index] -= quantity;
```



第二部：把資料換成資料庫

第二部 — 不換購物車邏輯，只換資料的來源



原本

資料放在 Java 陣列



現在

資料放在資料庫表格



商品名稱、價格、庫存這些「資料」不變；改變的是它們被保存、查詢的位置。

同一份資料，換一個家



今天用的：平行陣列

```
String[] products = {...};  
int[] prices = {...};  
int[] stock = {...};
```

簡單、好懂、但有上限



真實世界：資料庫

```
SELECT * FROM products  
WHERE stock > 0  
ORDER BY price;
```

強大、可搜尋、可擴展



關鍵操作：不是重新做一個系統，而是把 `products`、`prices`、`stock` 的資料來源換成 `products` 資料表。



第二部：真的操作資料庫

Step 2.1 — 準備 Java 的 SQLite 驅動

下載 SQLite JDBC Driver

1. 下載 [sqlite-jdbc-3.53.4.0.jar](https://www.sqlite.org/java.html)
2. 放到 Java 專案的 `lib/` 資料夾
3. Java 透過這個 JAR 連線 SQLite

這個 JAR 讓 Java 能使用 SQLite，不需要另外啟動資料庫伺服器。

SQLite 是單一檔案

- ✓ 不用安裝 MySQL Server
- ✓ 不用設定帳號和密碼
- ✓ `products.db` 就是完整資料庫
- ✓ Java 透過 JAR 讀取和修改它

🔗 SQLite 官網提供資料庫引擎；Java 課堂使用的 JDBC JAR 通常下載自 Xerial 的 `sqlite-jdbc` Releases。



用 VS Code 看資料庫

Step 2.2 — 安裝 SQLite Viewer , 直接看資料

安裝套件

1. 開啟 VS Code 的 Extensions
2. 搜尋 **SQLite Viewer**
3. 安裝作者為 **qwtel** 的套件

也可以在終端機執行：

```
code --install-extension qwtel.sqlite-viewer
```

打開資料

1. 在 VS Code 開啟 `products.db`
2. 點選資料表 `products`
3. 直接看到欄位和每一筆資料

這就是資料庫裡真正保存的資料。



Java JDBC 負責連線並執行 SQL ; SQLite Viewer 負責讓我們用表格畫面觀察資料。



建立商品資料表

Step 2.3 — CREATE TABLE

```
1 CREATE TABLE products (  
2     id INTEGER PRIMARY KEY,  
3     name TEXT NOT NULL,  
4     price INTEGER NOT NULL,  
5     stock INTEGER NOT NULL  
6 );  
7  
8 INSERT INTO products (name, price, stock) VALUES  
9     ('cola', 30, 10),  
10    ('sandwich', 65, 5),  
11    ('chips', 45, 8),  
12    ('cookie', 35, 12),  
13    ('juice', 40, 7);
```

欄位 **column**

id、name、price、stock

資料列 **row**

一筆商品資料

資料表 **table**

所有商品的集合

👁️ 讓資料庫把資料印出來

Step 2.4 — SELECT

```
1  -- 看全部商品
2  SELECT * FROM products;
3
4  -- 只看還有庫存的商品
5  SELECT name, price, stock
6  FROM products
7  WHERE stock > 0
8  ORDER BY price;
9
10 -- 找出 cola，準備檢查庫存
11 SELECT * FROM products WHERE name = 'cola';
```



在終端機執行查詢，或用 SQLite Viewer 打開資料表，都能看到後端程式平常讀取的資料。

你現在看見的是：一張有 id、name、price、stock 欄位的 products 表。



Java 連線 SQLite 完成購買

Step 2.5 — Java JDBC : 更新資料，再讀回結果

```
1  import java.sql.Connection;
2  import java.sql.DriverManager;
3  import java.sql.PreparedStatement;
4  import java.sql.ResultSet;
5
6  public class BuyProduct {
7      public static void main(String[] args) throws Exception {
8          int productId = 1;
9          int quantity = 2;
10
11          String updateSql = "UPDATE products "
12              + "SET stock = stock - ? "
13              + "WHERE id = ? AND stock >= ?";
14
15          try (Connection connection = DriverManager.getConnection("jdbc:sqlite:products.db")) {
16              // 先檢查庫存，再扣除購買數量
17              try (PreparedStatement statement = connection.prepareStatement(updateSql)) {
18                  statement.setInt(1, quantity);
19                  statement.setInt(2, productId);
20                  statement.setInt(3, quantity);
21
22                  if (statement.executeUpdate() > 0) {
```



編譯並執行 Java

Step 2.6 — 讓 Java 找得到 SQLite JAR

先確認資料夾

```
order-system/  
├─ BuyProduct.java  
├─ products.db  
└─ lib/  
    └─ sqlite-jdbc-3.53.4.0.jar
```

請在 `order-system/` 資料夾開啟終端機。

執行順序

1. 先編譯：產生 `BuyProduct.class`
2. 再執行：啟動 Java 程式
3. 最後用 Viewer 查看庫存

macOS / Linux

```
1 # 1. 編譯：-cp 告訴 Java 去 lib 找 SQLite JAR  
2 javac -cp "lib/sqlite-jdbc-3.53.4.0.jar" BuyProduct.java  
3  
4 # 2. 執行：. 代表目前資料夾，: 用來分隔兩個位置  
5 java -cp ".:lib/sqlite-jdbc-3.53.4.0.jar" BuyProduct.class
```



從陣列到資料庫

今天的學習路線

1. Java 陣列：先理解資料和邏輯
2. SQL 資料表：讓資料可以保存
3. Java JDBC：讓程式連線並操作資料庫

下一步會發生什麼？

Java 使用 JDBC 連線 `products.db`。

Java 傳入數量，執行庫存更新。

查詢結果回到 Java，再顯示給使用者。



今天的關鍵：陣列裡的資料，現在由 Java JDBC 從真正的 SQLite 資料庫讀取與修改。



DomJudge 題庫

線上練習 — 挑戰自我

登入系統

網址：domjudge.ntubimdbirc.tw

帳號：學號

密碼：預設密碼（已發Email，如找不到請找助教）



題目說明

- ✓ 題目從簡單到困難
- ✓ 用 Java 作答
- ✓ 提交後系統自動批改
- ✓ WA 是答案錯誤 RE是程式執行編輯錯誤



操作說明影片



小提示：先完成遊戲再來挑戰，題目不會很難！



恭喜！

你今天做出了一個會動的訂單系統

這就是後端的真實面貌



範例程式碼

LucasHsu.dev — 2026 商智中心後端群體驗營